A General Variable Neighborhood Search for Logistic Network Flow Design with Nonlinear Edge Costs and Transshipment Capacities

OPTIMA 2026

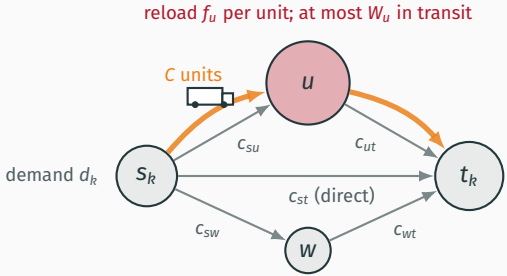
Varvara Kovaleva[1] Alexander Ponomarenko[2]

[1]HSE University, Nizhny Novgorod

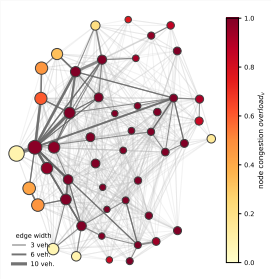
[2]Laboratory of Algorithms and Technologies for Network Analysis, HSE University, Nizhny Novgorod

The problem

Freight between regional sorting centres



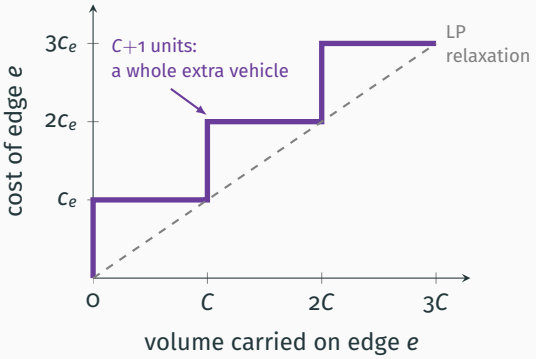
- **storages** v: reload cost f_v per unit, transshipment capacity W_v
- **roads** e: cost c_e per vehicle; a vehicle carries C units ($C = 50$)
- **requests** k: ship d_k units from s_k to t_k, reloading on the way if useful



Germany, $|V| = 49$: a GVNS solution. Node area $= W_v$, colour $=$ congestion (dark $=$ at capacity), thick edges carry ≥ 3 vehicles.

A MIP solver given two hours returns a solution 54% more expensive than this one.

Feature 1: a road is billed per truck, not per unit



schematic, $C = 50$

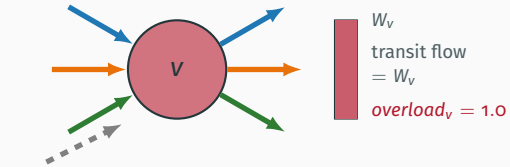
The billed quantity is the vehicle count

$$\text{cost of } e = c_e\, y_e, \qquad y_e \in \mathbb{Z}_+$$

with enough vehicles for the flow:
$\sum_k x_e^k \le C\, y_e$

- a non-convex *staircase* in the volume, shared by all requests on the edge
- dropping integrality gives the line $c_e x / C$: the LP relaxation, polynomial but *loose*

Feature 2: every storage has a reload ceiling



congestion of a storage in a solution:
$overload_v$ = transit flow through v / W_v
1.0 = at capacity

Transit volume through v is bounded

$$\sum_{k \in K:\, v \neq s_k, t_k} \; \sum_{e \in \delta^-(v)} x_e^k \;\leq\; W_v \qquad \forall v \in V$$

- once a storage is at capacity, further transit must detour: a longer route or a reload at a less loaded storage
- origin and destination handling is not transshipment: a direct $s_k \to t_k$ route touches no node capacity

Why not simply run a published network-design code?

	fixed-charge capacitated MCND	**this problem**
arc cost	fixed charge per opened arc + linear flow cost	c_e per vehicle: a staircase in the volume
capacities	per arc	per arc (through vehicles) **and** per node: transit volume $\leq W_v$
extra cost term	—	reload f_v per unit at every transit node
comparable published code	several (tabu, local branching, matheuristics)	none that we know of

Consequence for the experimental design

no published network-design code that we know of charges per vehicle and bounds node throughput, and our open corpus has no published objective values: we compare GVNS against **valid bounds** and **MIP solvers on the same model**

Model

The model: an arc-based MILP

$x_e^k \in \mathbb{R}_+$: flow of request k on edge e; $y_e \in \mathbb{Z}_+$: vehicles on e; $|K| \cdot |E| + |E|$ variables.

$$\min \sum_{e \in E} c_e\, y_e \;+\; \sum_{k \in K} \sum_{\substack{v \in V \\ v \neq s_k, t_k}} \sum_{e \in \delta^-(v)} f_v\, x_e^k \qquad \text{vehicles + reloads}$$

$$\text{s.t.} \sum_{e \in \delta^+(v)} x_e^k - \sum_{e \in \delta^-(v)} x_e^k = \begin{cases} d_k & v = s_k \\ -d_k & v = t_k \\ 0 & \text{otherwise} \end{cases} \qquad \forall k,\ \forall v \quad \text{(demand)}$$

$$\sum_{k \in K} x_e^k \leq C\, y_e \qquad \forall e \in E \quad \text{(vehicle capacity)}$$

$$\sum_{k \in K:\, v \neq s_k, t_k} \sum_{e \in \delta^-(v)} x_e^k \leq W_v \qquad \forall v \in V \quad \text{(transshipment capacity)}$$

$$x \geq 0, \quad y \in \mathbb{Z}_+^{|E|}$$

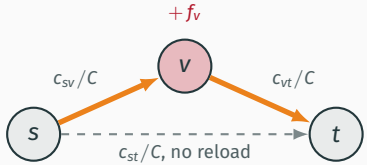
linear objective and constraints, integer y, continuous x: a MILP, NP-hard. This arc model is exactly what the exact solvers (HiGHS, SCIP) receive later.

Path form, and what the LP relaxation tells us

Path form. Flow x_p^k on paths $p \in \Pi_k$ (all $s_k \to t_k$ paths), $\sum_{p \in \Pi_k} x_p^k = d_k$, same y_e; only the paths that carry flow matter.

Relax integrality. The capacity constraint becomes tight, so $y_e = \frac{1}{C} \sum_k \sum_{p \ni e} x_p^k$ and y disappears:

$$\min \sum_{k \in K} \sum_{p \in \Pi_k} x_p^k \underbrace{\left(\frac{1}{C} \sum_{e \in p} c_e + \sum_{\substack{v \in p \\ v \neq s_k, t_k}} f_v \right)}_{\text{per-unit price of path } p}$$



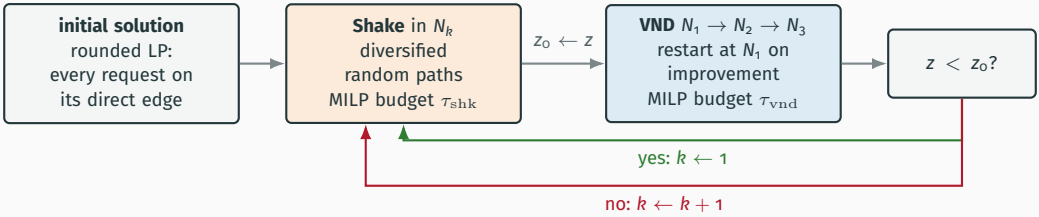
Reading

a unit of flow pays *travel*$/C$ plus one reload per transit node: the relaxation prefers cheap, transit-free paths

- on the completed graph the relaxed optimum puts every request on its direct edge: our **starting solution**; the search metric of N_1 is an edge-wise version of this price

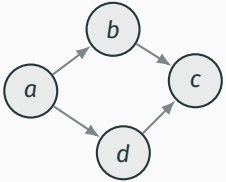
The GVNS

GVNS at a glance

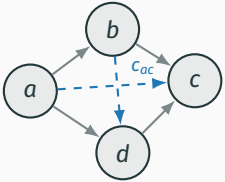


- **search space: commodity paths.** A solution is a path pool P with flows x; vehicle counts follow, $y_e = \left\lceil \frac{1}{C} \sum_k x_e^k \right\rceil$
- **shake and VND are the same primitive** (next slide): the shake adds more, random paths with a larger MILP budget
- **stop** when $k > 3$ or after $t_{\max} = 2|V|$ minutes; a move is accepted *only if it improves*, so the shake never degrades the incumbent, and k advances when the VND fails to beat the post-shake value z_0

Preprocessing and the starting solution



(a) raw road graph



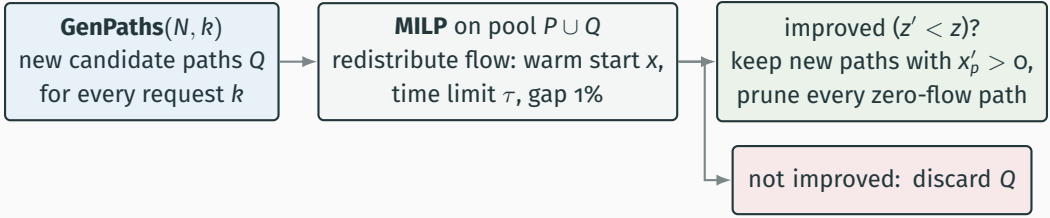
(b) completed graph

- road graphs are sparse ($a \to c$ needs a stop); on the raw graph a missing origin–destination path can even make an instance infeasible
- **completion**: join every ordered pair by an edge costing the shortest path in G (c_{ac}): the cheapest *stop-free* route, touching no W_v
- the exact solvers get the same completed graph ($|E| = |V|(|V| - 1)$)

Start: the rounded LP

every request on its direct edge (the relaxed optimum), vehicles rounded up: always feasible, and poor (1.19× GVNS at $|V| = 10$, 2.80× at 50, 3.14× on European Russia)

The move primitive: propose paths, let the MILP redistribute

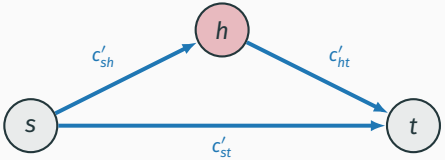


Why paths and not edge flows

per-edge flows are strongly coupled across requests, so a feasible change needs the whole network at once; a path describes one request only, and *adding* a path to a feasible solution never makes it worse

- the three neighbourhoods differ *only* in **GenPaths**; everything else is shared
- pruning zero-flow paths keeps the in-loop MILP small enough for budgets of 25–70 s (VND) and $10+|V|$ to $100+2|V|$ s (shake)
- the neighbourhood proposes, the MILP disposes

N_1: reload-aware shortest paths



Auxiliary complete graph G': per-unit weights

$$c'_{vu} = f_v + f_u + \frac{c_{vu}}{C} \qquad \forall (v,u) \in E$$

travel per unit plus the reloads at the endpoints; plain shortest paths would ignore the reloads

- 20 shortest paths per request under c', precomputed once; new paths come from this pool
- **VND**: $\{1,2,3\}$ new paths per request $\times$ {random, low-cost first}; **Shake**: 5 random pool paths
- ablation: without N_1, $+0.8\%$ (median, synthetic and real alike)

N_2: fill the half-empty trucks

edge (u, v): 110 units, $y_{uv} = \lceil 110/50 \rceil = 3$



$load_{uv} = 110 \bmod 50 = 10$:
up to 40 more units here cost **no extra vehicle** and may free one elsewhere

- shortest paths on G'' run through partially loaded vehicles
- dynamic number of new paths: stop after 2 fully loaded paths or free room $> 1C$ (VND); 3 paths / $5C$ (Shake)

Graph G'' favours free room ($\alpha = 0.5$)

$$c''_{vu} = f_v + f_u + \frac{load_{vu}}{C}\left(\frac{c_{vu}}{C}\right)^{\alpha}$$

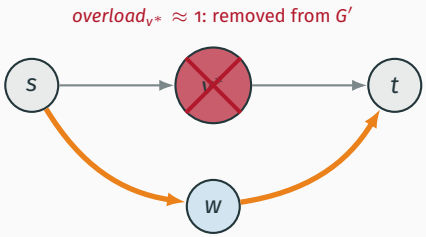
$$load_e = \left(\sum_k \sum_{p \ni e} x_p^k\right) \bmod C$$

fully loaded edges ($load_e = 0$) keep their G' weight; smaller residual load $\Rightarrow$ lower weight

Ablation: the dominant operator

without N_2: $+3.9\%$ synthetic, $+1.6\%$ real (medians), up to $+23.8\%$ on a single instance

N_3: detour around the saturated storage



new shortest path through a less loaded w

in the current solution:

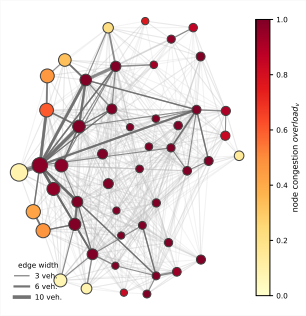
$$overload_v = \frac{1}{W_v} \sum_{k:\, v \neq s_k, t_k} \; \sum_{p \in \Pi_k :\, v \in p} x_p^k$$

- delete the **single most congested** storage from G' (unless it is the request's own origin or destination), recompute the shortest path: a detour to less loaded storages
- 1 new path per request in VND, 3 in Shake
- ablation: +0.1% / +0.2% median, **inactive when capacities are slack**; +4.0% on Germany, where almost every storage runs at capacity

Experiments

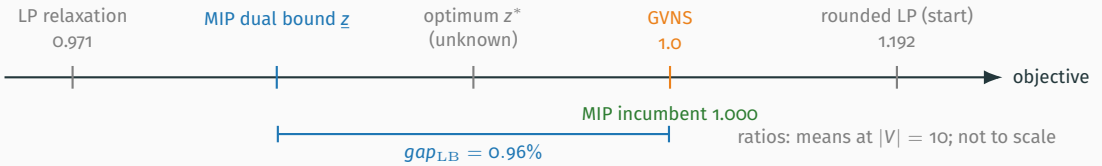
The benchmark: realistic regional networks, openly available

- our open corpus (arXiv:2606.20653; Zenodo, CC-BY-4.0), every instance reproducible from its published seed
- road networks between regional sorting centres, calibrated on OpenStreetMap for **nine European countries, five U.S. states and European Russia**
- real structure: Zipf city sizes, spatial-network topology, centrality-driven W_v, gravity demand; near-planar, large-diameter graphs
- **35 synthetic**: $10 \times |V|$=10, 7×20, 7×30, 5×50, 3×100, 3×150
- **15 real-geometry**: Denmark ($|V| = 11$) to European Russia ($|V| = 90$)



Germany ($|V| = 49$), the solution from the first slide: almost every storage runs at capacity.

Setup and yardsticks



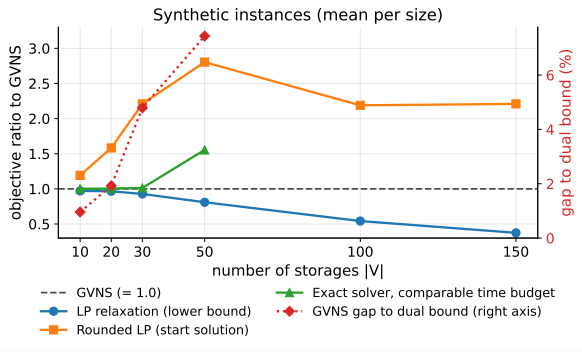
Two metrics

$gap_{\mathrm{LB}} = (Result_{\mathrm{GVNS}} - \underline{z})/\underline{z}$, $\underline{z}$ = best MIP dual bound: an **upper bound on the true optimality gap** of GVNS

$Ratio_{\mathrm{method}} = Result_{\mathrm{method}}/Result_{\mathrm{GVNS}}$: below 1 better than GVNS, above 1 worse

- GVNS: 10 seeds per instance, limit $2|V|$ min; exact: gap target 0, 30 min–4 h, i.e. comparable budgets ($|V| = 50$: 100 min vs 2 h); HiGHS is the exact baseline *and* the in-loop solver
- nothing is proven optimal, but $\underline{z}$ is a valid lower bound, typically within a few percent of the incumbent: gap_{LB} is a guarantee, not a self-comparison

Results I: synthetic instances



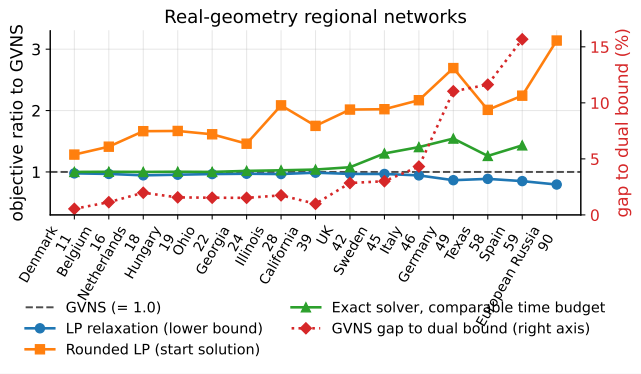
$\lvert V\rvert$	n	gap_{LB}	exact inc.	σ
10	10	0.96%	1.000	0.06%
20	7	1.92%	1.005	0.18%
30	7	4.79%	1.013	0.68%
50	5	7.44%	1.555	0.66%
100	3	†	†	0.31%
150	3	†	†	0.38%

†: arc model intractable; only the loose LP bound (0.542, 0.375)

near-optimal up to $|V| = 20$; at $|V| = 50$ the exact solver with a comparable budget is 55% more expensive on average (16% to 137% per instance)

GVNS mostly stops before its limit (3 729 of 6 000 s at $|V| = 50$); the exact solver always used its full budget, so the comparison is conservative.

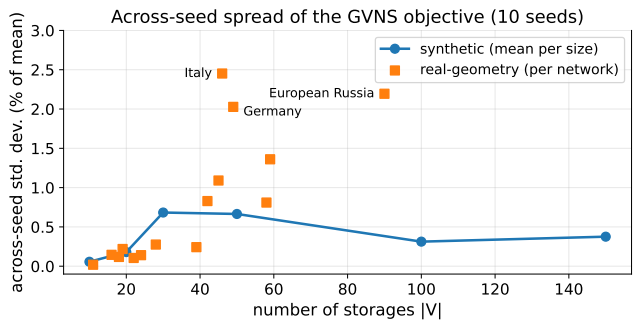
Results II: real road geometries mirror the synthetic story



- exact solver with a comparable budget: parity up to $|V| = 22$, 1.02–1.08× up to the UK (42), then 1.26–1.54× from Sweden (45) to Spain (59)
- gap_{LB} from 0.5% (Denmark, 11) to 15.7% (Spain, 59)
- European Russia ($|V| = 90$): no exact reference; GVNS ends 3.1× below its start

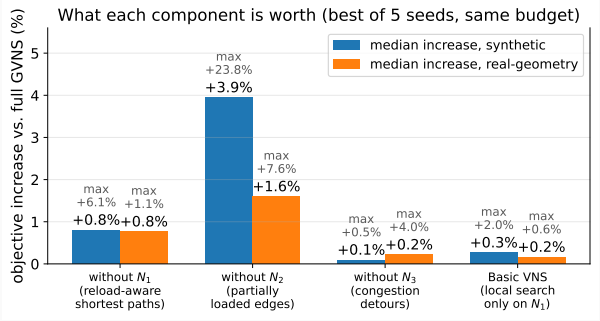
same shape as the synthetic curves: the synthetic corpus behaves like the real geometries, and GVNS is robust to them

How many seeds do you need?



- synthetic: across-seed σ below 0.7% of the mean at every size, below 0.2% up to $|V| = 20$
- large real networks: 2.0% (Germany), 2.5% (Italy), 2.2% (European Russia)
- three seeds badly understate it: on Italy the spread over three seeds is 0.4% against 2.5% over ten
- one run is representative on small networks; on capacity-tight instances take the best of several

What each component buys



best of 5 seeds, same budget, subset of synthetic ($|V| = 20$–100) and real (16–49) instances; medians because the seed spread is heavy-tailed on capacity-tight networks

GVNS better / tie / variant better	synth.	real
without N_1	8 / 0 / 1	5 / 1 / 0
without N_2	8 / 1 / 0	5 / 1 / 0
without N_3	1 / 8 / 0	3 / 3 / 0
Basic VNS	5 / 4 / 1	2 / 4 / 0

"decided" = difference beyond the seed spread of both runs; time relative to GVNS: 0.80, 0.52, 0.68, 0.63

- N_2 dominates; N_3 is situational: ties when capacities are slack, $+4.0\%$ on Germany
- **Basic VNS** (local search on N_1 only, same shake): GVNS better on 7 of 16, Basic VNS on 1, 8 ties; the margin grows with size, 0.2–0.6% at $|V| \leq 28$ vs 1.1–2.0% at $|V| \geq 50$, for $1/0.63$ of the time

Closing

Limitations, and what comes next

Stated before you ask

- **static capacities and costs**: on real networks they vary with the time of day
- **no proven optima**: gap_{LB} is an upper bound on the true gap
- **no tight reference beyond** $|V| = 59$: only the loose LP bound
- **path generation is heuristic**: a pool of 20 shortest paths plus three rules
- **open-source solvers at default settings** as the exact baseline (tuning checked on 8 instances)
- **component study** on a subset with five seeds; **no operational data** yet

Next

- time-expanded graph with period-dependent $C_e(t)$, $W_v(t)$: the neighbourhoods only need a period-aware edge metric
- column-generation pricing to feed the path pool
- validation on operational data

Takeaways

1. integer trucks + reload ceilings: compare against **valid dual bounds**, not fixed-charge codes
2. path-space GVNS: within $\approx 1\%$ of the dual bound on small networks; the exact solver is **up to 55% more expensive** at $|V| = 50$
3. N_2 is the lever, N_3 only when capacity binds, the full VND pays at scale

Let cheap path generators propose and a time-boxed MILP dispose: N_2 always, N_3 when capacity binds, the full VND once the network is large.

Thank you

Code, exact-solver harness, scripts that regenerate every table and figure
`github.com/aponom84/Optimal-Logistic-Net`

Instance corpus (CC-BY-4.0)
`doi.org/10.5281/zenodo.20372456`

Contact
`aponomarenko@hse.ru` ·
`varvara_kovaleva03@mail.ru`

Supported by the Basic Research Program at HSE University (HSE-BR-2025-080) and by the HPC facilities of HSE University (cHARISMa).

code

data

Backup: settings (Table 1 of the paper)

Setting	Value	Setting	Value
neighbourhoods $k_{\max}$	3	N_2 exponent α	0.5
GVNS limit $t_{\max}$	$2\|V\|$ min	N_2 free-room stop	$1C$ / $5C$
random seeds	10 / 5 (components)	N_3 storages removed	1 (most congested)
vehicle capacity C	50	N_1 path pool	20
N_1 paths per request	1–3 / 5	N_3 paths per request	1 / 3
initial MILP limit	300 s	exact gap target	0
redistribution MILP gap	1%	exact budget	30 min–4 h
τ_{shk} (N_1 / rest)	$10+\|V\|$ / $100+2\|V\|$ s	τ_{vnd} (N_1 / rest)	25 / 70 s (15 / 50 s at $\|V\| = 100$)

Values given as VND / Shake where they differ. Each job pinned to 8 cores; HSE cluster cHARISMa ($2\times$AMD EPYC, 44 cores, $\geq$720 GB RAM per node); HiGHS 1.14.0, SCIP 10.0.2.

Backup: the exact side, and is the baseline understated?

	HiGHS 1.14	SCIP 10	arc MILP size
$\lvert V\rvert \leq 42$	dual bound + incumbent	dual bound agrees with HiGHS within 0.6% (mean 0.18%); weaker incumbent	$\approx$ 6 000 variables at $\lvert V\rvert = 10$
$\lvert V\rvert = 45$–59	dual bound + incumbent	no feasible solution, no non-trivial bound	
$\lvert V\rvert \geq 90$	millions of variables: at $\lvert V\rvert = 90$ only SCIP was tried and returns no bound and no incumbent; out of memory for both at $\lvert V\rvert \geq 100$		3.7×10^8 at $\lvert V\rvert = 150$

budgets: MIP-gap target 0, 30 min ($\lvert V\rvert = 10$) to 4 h ($\lvert V\rvert = 150$); none of the 50 instances solved to proven optimality, but the branch-and-bound dual bound is tight and well above the LP.

Tuning check: 8 instances ($\lvert V\rvert = 30$–59), three non-default presets per backend (HiGHS: aggressive heuristics, presolve off, full parallelism; SCIP: aggressive heuristics, feasibility emphasis, aggressive separation). SCIP: no preset helps; two presets *degrade* the dual bound by up to 2.3% and the incumbent by up to 48%; on the six subset instances with $\lvert V\rvert \geq 45$ no preset finds a feasible solution.

The default configuration does not understate the exact baseline.

instance	HiGHS, column-wise best of 3 presets vs default	
	dual bound	incumbent
30_nodes_1	+0.005%	+0.054%
30_nodes_2	+0.015%	0
50_nodes_1	+0.002%	+0.285%
50_nodes_2	0	0
Sweden (45)	+0.003%	+0.823%
Germany (49)	+0.023%	+0.108%
Texas (58)	+0.036%	+0.454%
Spain (59)	+0.029%	+0.885%

+ = improvement; full grid in `Results/exact_tuning.csv`

Backup: what the means hide at $|V| = 50$

instance	gap_{LB}	exact inc.	rounded LP	LP relax.	mean GVNS time
50_nodes_1	2.99%	1.162	2.151	0.827	2 890 s
50_nodes_2	14.61%	2.368	4.131	0.777	5 583 s
50_nodes_3	5.25%	1.410	2.707	0.784	4 001 s
50_nodes_4	9.69%	1.523	2.668	0.854	2 954 s
50_nodes_5	4.64%	1.311	2.363	0.805	3 215 s
mean	7.44%	1.555	2.804	0.810	3 729 s

- 50_*nodes*_2 is where the exact solver falls furthest behind of any instance with a dual bound: its incumbent is 2.37× GVNS, the rounded-LP start 4.13×; only Spain ($|V| = 59$, 15.7%) has a larger gap_{LB}
- on this instance the 14.6% cannot be split between heuristic error and bound weakness; the LP bound alone would put the gap at 29%, so the dual bound is already much tighter
- per-instance gap_{LB} at the smaller sizes: 0.38–1.95% at $|V| = 10$, 1.08–2.66% at 20, 2.42–6.58% at 30

Backup: where it sits in the literature

Capacitated multicommodity network design

- Gendron, Crainic, Frangioni (1999); Crainic (2000): fixed-charge MCND, NP-hard
- Barnhart et al. (1998): branch-and-price on path formulations
- Ghamlouche, Crainic, Gendreau (2003): cycle-based tabu search
- Rodríguez-Martín, Salazar-González (2010): local branching
- Hewitt, Nemhauser, Savelsbergh (2010): MILP-in-the-loop matheuristics
- Crainic, Gendreau (2021): survey of (meta)heuristics

Step costs and node capacities

- Gabrel, Knippel, Minoux (1999): exact Benders-type method for general step costs
- Croxton, Gendron, Magnanti (2003): MILP models of nonconvex piecewise-linear costs
- Alumur, Kara (2008): hub location with bounded hub throughput

Variable neighbourhood search

- Mladenović, Hansen (1997); Hansen et al. (2017); Hansen, Mladenović (2019): VNS, GVNS
- Karakostas et al. (2019); Mjirda et al. (2016): GVNS vs single-neighbourhood schemes
- Masri, Krichen, Guitouni (2015): multi-start VNS for the single-path multicommodity flow problem
- Amous et al. (2017): VNS for the capacitated VRP

Real-life logistic routing

- Batsyn, Ponomarenko (2014, 2015): truck-and-trailer routing with time windows and split deliveries

Benchmark

- Kovaleva, Ponomarenko (2026): generator and corpus, arXiv:2606.20653, Zenodo 10.5281/zenodo.20372456